

Capitolul 1

Operații cu Fișiere și Directoare

mkdir, dd, truncate, du, tree, zip, tar

Ce trebuie să știi — teoria

În Linux, totul este un fișier. Directoarele sunt de fapt fișiere speciale care conțin referințe către alte fișiere. Comenzile de bază pentru navigare și creare sunt esențiale și apar aproape la orice probă practică.

1a. Crearea ierarhiei de directoare

mkdir (make directory) creează directoare. Opțiunea **-p** (parents) creează automat toate directoarele părinte care lipsesc, fără erori dacă există deja. Fără **-p**, dacă directorul părinte nu există, primești eroare.

Sintaxa cu **acolade { }** se numește **brace expansion** și e o scurtătură bash care generează automat mai multe cai. De exemplu: **calc/{9-10,11-12}** se expandează la **calc/9-10 calc/11-12**. Poți imbrica acoladele oricum vrei.

Terminal – creare ierarhie

```
# Comanda completa cu brace expansion:
mkdir -p AcadNet/{calc/{9-10,11-12},is/{9-10,11-12},rl/{9-10,11-12}}

# Ce face exact:
# Creeaza AcadNet/calc/9-10
# Creeaza AcadNet/calc/11-12
# Creeaza AcadNet/is/9-10 etc.

# Verificare - doua metode:
tree AcadNet # afiseaza arbore vizual
find AcadNet -type d | sort # listeaza doar directoare, sortat
```

■ TIP: Dacă 'tree' nu e instalat: `sudo apt install tree`

1b. Crearea a 50 fișiere cu dimensiune fixă

Există două utilitare principale pentru creare de fișiere cu dimensiune prestabilită:

Utilitar	Viteza	Conținut	Când să-l folosești
dd	Mediu	/dev/zero = zeros /dev/urandom = random	Cel mai comun la olimpiade
truncate	Foarte rapid	Fișier 'sparse' (gol aparent)	Dacă se cere doar dimensiunea
fallocate	Rapid	Date nedefinite	Alternativa la truncate

Anatomia comenzii **dd**:

Creare 50 fisiere de 12MB

```
dd if=/dev/urandom of=AcadNet/calc/9-10/subiect_1 bs=1M count=12
| | | | |
if=input file of=output file(destinatie) | | count=numarul de blocuri
bs=block size(1M=1 megabyte)

# 1M * 12 = 12MB -- asta e dimensiunea finala

# Pentru toate 50 fisierele, folosim un for loop:
for i in $(seq 1 50); do
dd if=/dev/zero of=AcadNet/calc/9-10/subiect_$i bs=1M count=12 status=none
done

# status=none suprima mesajele de progres ale dd
# $(seq 1 50) genereaza numerele 1 2 3 ... 50
# $i este variabila de iteratie

# Verificare:
ls -lh AcadNet/calc/9-10/ | head
du -sh AcadNet/calc/9-10/subiect_1 # trebuie sa arate ~12M
```

■ **DE RETINUT:** *Daca se cer 'date aleatoare', foloseste /dev/urandom. Daca nu se specifica, /dev/zero e mai rapid.*

1c. Afisarea dimensiunii totale si structurii

Afisare dimensiune si structura

```
# Dimensiunea totala a ierarhiei:
du -sh AcadNet/
# -s = summarize (total, nu fiecare fisier)
# -h = human readable (MB, GB, in loc de bytes)

# Structura arborescenta (ordonata):
tree AcadNet

# ATENTIE: ls -l sorteaza alfabetic: subiect_10 vine inainte de subiect_2!
# Pentru sortare NUMERICA corecta:
ls -v AcadNet/calc/9-10/ # -v = version sort (natural numeric)
find AcadNet -print | sort -V # sort -V face acelasi lucru
```

■ **ATENTIE:** *Sortarea implicita e lexicografica: subiect_10 < subiect_2 < subiect_20. Foloseste -v sau sort -V!*

1d. Arhiva ZIP — creare si afisare continut

Linux are doua sisteme principale de arhivare: **zip** (clasic Windows-compatibil) si **tar+gzip** (nativ Unix). La proba, citeste cerinta cu atentie — specifica extensia (.zip sau .tar.gz).

Arhivare ZIP si TAR

[illegible]

Gestionarea Proceselor si Planificarea

ps, renice, at, cron, kill, jobs, sleep

Ce este un proces?

Un **proces** este un program aflat in executie. Fiecare proces are un **PID** (Process ID) unic. Procesele au un **owner** (utilizatorul care l-a pornit), o **prioritate** (nice value) si consuma **CPU** si **memorie**.

2a. Top 5 procese dupa memorie + renice

ps aux afiseaza toate procesele. Coloanele importante:

Coloana	Semnificatie
USER	utilizatorul proprietar al procesului
PID	identificatorul unic al procesului
%CPU	procentul din CPU folosit
%MEM	procentul din RAM folosit
COMMAND	comanda care a pornit procesul
NI	nice value (prioritatea)

renice schimba prioritatea unui proces care ruleaza deja. **Nice value**: de la -20 (cea mai mare prioritate) la 19 (cea mai mica). Valoarea implicita e 0. Cu cat e mai mic numarul, cu atat procesul primeste mai mult CPU.

Renice top 5 procese

```
# Pasul 1: Afiseaza toate procesele userului curent, ordonate dupa %MEM:
ps aux --sort=-%mem | grep ^$(whoami)
# --sort=-%mem = descrescator dupa memorie (- inseamna descrescator)
# grep ^$(whoami) = filtreaza liniile care incep cu username-ul tau
# $(whoami) = comanda care returneaza username-ul curent

# Pasul 2: Ia doar primele 5:
ps aux --sort=-%mem | grep ^$(whoami) | head -5

# Pasul 3: Extrage PID-urile (coloana 2):
ps aux --sort=-%mem | grep ^$(whoami) | head -5 | awk '{print $2}'

# Pasul 4: Renice fiecare PID la 15:
for pid in $(ps aux --sort=-%mem | grep ^$(whoami) | head -5 | awk '{print $2}'); do
renice -n 15 -p $pid
done

# Alternativa cu xargs (one-liner):
ps aux --sort=-%mem | grep ^$(whoami) | head -5 | awk '{print $2}' | xargs -I{} renice -n 15 -p {}

# Verificare - NI trebuie sa fie 15:
ps aux --sort=-%mem | grep ^$(whoami) | head -5
```

■ **DE RETINUT:** Pentru nice values negative (prioritate mai mare), ai nevoie de sudo. Pentru valori pozitive (15) nu e nevoie.

2b. Planificarea comenzilor cu 'at'

at executa o comanda o singura data la un moment specificat in viitor. Spre deosebire de **cron** (care repeta periodic), **at** e pentru executii unice.

Programare cu at

```
# Sintaxa de baza:
echo 'comanda' | at ORA DATA

# Exemple de formate acceptate:
echo 'echo "Hello from AcadNet 2025"' | at midnight August 1 2025
echo 'echo "Hello from AcadNet 2025"' | at 00:00 08/01/2025
echo 'echo "Hello from AcadNet 2025"' | at 00:00 Aug 1, 2025

# Alternativ cu heredoc:
at 00:00 August 1 2025 <<EOF
echo "Hello from AcadNet 2025"
EOF

# Administrare joburi at:
atq # listeaza joburile programate
atrm 1 # sterge job-ul cu numarul 1
at -c 1 # afiseaza continutul job-ului 1

# Daca 'at' nu e instalat:
sudo apt install at -y
sudo systemctl enable --now atd
```

■ **TIP:** La concurs, dupa ce rulezi comanda **at**, ruleaza '**atq**' ca sa dovedesti supraveghetorului ca job-ul e programat.

2c. 10 procese sleep + SIGKILL (one-liner)

Semnalele Linux permit comunicarea cu procesele. Cele mai importante:

Semnal	Nr	Efect
SIGTERM	15	Cerere de terminare gradata (procesul poate ignora)
SIGKILL	9	Terminare fortata imediata (nu poate fi ignorata)
SIGHUP	1	Reload configuratie (de obicei)
SIGSTOP	19	Suspenda procesul (pauzare)
SIGCONT	18	Reia un proces suspendat

One-liner 10 sleep + SIGKILL

```
# Metoda 1 – porneste + asteapta un pic + pkill:
for i in $(seq 1 10); do sleep 30 & done; sleep 0.5; pkill -SIGKILL sleep

# Metoda 2 – retine PID-urile si trimite KILL direct:
PIDS=(); for i in $(seq 1 10); do sleep 30 & PIDS+=(!); done; kill -9 ${PIDS[@]}
# $! = PID-ul ultimului proces pornit in background
# ${PIDS[@]} = toate elementele array-ului

# Metoda 3 – cu jobs -p:
for i in $(seq 1 10); do sleep 30 & done; kill -KILL $(jobs -p)

# Verificare ca au murit:
jobs
ps aux | grep sleep
```

2d. Proces sleep cu nume personalizat

În mod normal, comanda **sleep** apare în lista proceselor ca 'sleep'. Prin **exec -a 'nume'** putem da un alt nume procesului, care va apare în ps/top cu acel nume.

Proces cu nume custom

```
# Creeaza un proces sleep cu un nume personalizat:
(exec -a "de ce rinocerul are corn?" sleep 1000) &
# ( ) = subshell
# exec -a 'nume' = executa urmatoarea comanda cu numele dat
# & = in background

# Verificare:
ps aux | grep "rinocerul"
pgrep -a "rinocerul" # mai elegant

# Oprește:
pkill -f "rinocerul"
```

Personalizarea Terminalului

PS1, culori ANSI, cowsay, cursor, nano ~/.bashrc

Cum functioneaza PS1 si ~/.bashrc

PS1 (Prompt String 1) este variabila bash care defineste cum arata promptul terminalului — adica linia pe care apare userul, hostname-ul si directorul curent. Modificarile in **~/.bashrc** sunt permanente si se aplica la fiecare sesiune noua. Dupa modificare, trebuie sa rulezi **source ~/.bashrc** ca sa se aplice imediat.

3a. Colorare PS1 — ghid complet coduri ANSI

Culorile in terminal se seteaza prin **secvente escape ANSI**. Sintaxa generala este: **\[033[TRIBm\]** unde **TRIB** e un numar sau mai multe numere separate prin **;**. Parantezele **\[** si **\]** spun bash-ului ca aceste caractere nu ocupa spatiu vizibil (important pentru calculul lungimii liniei).

Cod	Culoare text	Vizual	Echivalent fundal (+10)
30	Negru		40 = fundal negru
31	Rosu		41 = fundal rosu
32	Verde		42 = fundal verde
33	Galben		43 = fundal galben
34	Albastru		44 = fundal albastru
35	Magenta		45 = fundal magenta
36	Cyan		46 = fundal cyan
37	Alb		47 = fundal alb
90	Gri (bright black)		100 = fundal gri
91	Rosu deschis		101 = fundal rosu deschis
92	Verde deschis		102 = fundal verde deschis
93	Galben deschis		103 = fundal galben deschis
94	Albastru deschis		104 = fundal albastru deschis
95	Magenta deschis		105 = fundal magenta deschis
96	Cyan deschis		106 = fundal cyan deschis
97	Alb stralucitor		107 = fundal alb stralucitor

Atribute de formatare (se pot combina cu **;**):

Cod	Efect
0	Reset — anuleaza toate atributele
1	Bold (text ingrosit)
2	Dim (text mai sters)
4	Underline (subliniat)

5	Blink (clipitor — nu pe toate terminalele)
7	Reverse (inverseaza culoare text/fundal)
9	Strikethrough (taiat)

Variabile speciale PS1 (ce poti pune in prompt):

Variabila	Se inlocuieste cu
\u	username-ul utilizatorului curent
\h	hostname (pana la primul '.')
\H	hostname complet (FQDN)
\w	directorul curent complet (ex: /home/student/AcadNet)
\W	doar numele directorului curent (ex: AcadNet)
\\$	\$ pentru useri normali, # pentru root
\t	ora curenta (HH:MM:SS)
\d	data curenta (Mon Jan 01)
\n	newline (linie noua in prompt)
\$(comanda)	output-ul comenzii (executata la fiecare prompt)

Cum se assembleaza o secventa PS1:

\\[\033[1;31m\\] = deschide culoare: 1=bold, 31=rosu, m=sfarsit cod, \\[\\] = delimitatori bash

Anatomia culorilor PS1

```
# Structura generala a unui cod de culoare:
\\[\033[ATRIB;CODm\\]TEXT_COLORAT\\[\033[0m\\]
|_____| |_____| |_____|
deschide inchide reset

# Cerinta: user=rosu bold, @=verde bold, host=galben bold
# user = \\[\033[1;31m\\]\u
# @ = \\[\033[1;32m\\>@
# host = \\[\033[1;33m\\)\h\\[\033[0m\\]:\w\$ '
# reset = \\[\033[0m\\]
```

Pasii pentru modificare permanenta cu nano:

1 Deschide ~/.bashrc cu nano:

```
nano ~/.bashrc
```

2 Mergi la sfarsitul fisierului:

Apasa Ctrl+End (sau End+End)

3 Adauga linia PS1:

```
PS1='\\[\033[1;31m\\]\u\\[\033[1;32m\\]@\\[\033[1;33m\\)\h\\[\033[0m\\]:\w\$ '
```

4 Salveaza fisierul:

Ctrl+O → Enter (confirma numele)

5 Iesi din nano:

```
Ctrl+X
```

6 Aplica modificarile:

```
source ~/.bashrc
```

■ **DE RETINUT:** Ghilimelele conteaza! Foloseste ghilimele simple ' ' in jurul PS1. Cu ghilimele duble, bash interpreteaza secventele imediat si e mai complicat.

3b. cowsay cu Hello Kitty

cowsay este un utilitar care afiseaza un mesaj intr-un 'balon' de vorbire, cu un personaj sub el. Vine cu mai multe personaje (fisiere .cow). Optiunea **-f** specifica personajul.

```
cowsay Hello Kitty

# Verifica daca cowsay e instalat:
cowsay --version
sudo apt install cowsay -y # instaleaza daca lipseste

# Listeaza toate personajele disponibile:
cowsay -l
ls /usr/share/cowsay/cows/ # sau din filesystem

# Cauta Hello Kitty:
cowsay -l | grep -i kitty
ls /usr/share/cowsay/cows/ | grep -i kitty

# Afiseaza Hello Kitty (daca e disponibila):
cowsay -f hellokitty "ACADNET 2025"

# Daca nu exista hellokitty, incearca variante:
cowsay -f kitty "ACADNET 2025"
cowsay -f cat "ACADNET 2025"
cowsay "ACADNET 2025" # default = vaca
```

3c. IP in loc de hostname in PS1

In loc de **lh** (hostname) punem o **comanda dinamica** care ruleaza la fiecare prompt si afiseaza IP-ul curent. Comanda **hostname -I** returneaza toate adresele IP, si luam prima cu **cut** sau **awk**.

PS1 cu IP dinamic

```
# Test — ce afiseaza hostname -I:
hostname -I # ex: 192.168.1.10 10.0.0.1
hostname -I | awk '{print $1}' # doar primul IP: 192.168.1.10
hostname -I | cut -d' ' -f1 # echivalent cu cut

# Deschide .bashrc:
nano ~/.bashrc

# Adauga la sfarsit (cu ghilimele simple!):
PS1='\[\033[1;31m\]\u\[\033[1;32m\]@\[\033[1;33m\]$(hostname -I | cut -d -f1)\[\033[0m\]:\w\$ '

# Salveaza (Ctrl+O, Enter) si iese (Ctrl+X), apoi:
source ~/.bashrc

# Alternativa cu ip addr:
ip addr show | grep 'inet ' | grep -v '127.0.0.1' | awk '{print $2}' | cut -d/ -f1 | head -1
```

■ TIP: Dupa `source ~/.bashrc`, promptul ar trebui sa arate ceva de genul: `student@192.168.1.10:~$`

3d. Cursor subliniat clipitor

Tipul cursorului se schimba cu secvente escape speciale trimise terminalului. Acestea nu sunt secvente de culoare (`\033[.m`) ci secvente de control al cursorului (`\033[.q`).

Cod	Tip cursor
<code>\033[0 q</code>	Default al terminalului
<code>\033[1 q</code>	Bloc clipitor
<code>\033[2 q</code>	Bloc solid (fara clipire)
<code>\033[3 q</code>	Subliniat CLIPITOR <-- cel cerut
<code>\033[4 q</code>	Subliniat solid
<code>\033[5 q</code>	Bara verticala clipitoare ()
<code>\033[6 q</code>	Bara verticala solida

Pasi:

Setare cursor subliniat clipitor

```
# Test imediat (fara permanenta):
echo -e "\033[3 q"
# sau:
printf "\e[3 q"

# Pentru a fi permanent — adauga in ~/.bashrc:
nano ~/.bashrc

# Adauga la sfarsit:
echo -e "\033[3 q"
# sau:
printf "\e[3 q"

# Salveaza (Ctrl+O → Enter → Ctrl+X) si aplica:
source ~/.bashrc
```

Utilizarea Makefile

Compilare C, reguli build/run/clean/pack, fisiere obiect

Ce este Makefile si de ce e util?

make este un utilitar care automatizeaza procesul de compilare. In loc sa rulezi manual **gcc** de fiecare data, definesti regulile o singura data in **Makefile** si apoi rulezi simplu **make** sau **make regula**. Makefile-ul stie ce trebuie recompilat (prin timestamp-uri) si nu recompileaza ce nu s-a schimbat.

REGULA DE AUR: Indentarea in Makefile se face EXCLUSIV cu TAB! Spatiile nu functioneaza si sunt o greseala clasica.

Structura Makefile

```
# Structura unui Makefile:

variabila = valoare # definitie variabila

tinta: dependente # regula: tinta depinde de...
[TAB] comanda_de_executat # corpul regulii - TREBUIE TAB!
[TAB] alta_comanda

# Exemplu concret:
CC = gcc
TARGET = acadnet2025

build: main.c firststep.c
[TAB] $(CC) -o $(TARGET) main.c firststep.c
```

4a. Regula build simpla

Makefile 4a

```
# Makefile complet pentru 4a:
CC = gcc
TARGET = acadnet2025

build:
[TAB]$(CC) -o $(TARGET) main.c firststep.c
[TAB]./$$(TARGET)

# Rulare:
make build
# sau simplu 'make' daca build e prima regula
make
```

4b. Compilare cu .o + run + clean

Un **fisier obiect (.o)** este cod compilat dar neligat (unlinked). Compilarea are doua etape: 1) sursa .c → obiect .o (cu -c), 2) obiect(e) .o → executabil (linking). Avantajul: daca schimbi un singur .c, recompilezi doar acel .o, nu tot.

Makefile 4b

```
CC = gcc
TARGET = acadnet2025

build:
[TAB]$(CC) -c zeus.c -o zeus.o # compileaza zeus.c -> zeus.o
[TAB]$(CC) -o $(TARGET) zeus.o secondstep.o # link zeus.o + secondstep.o
[TAB]./$$(TARGET)

# Daca zeus.o e dat deja (nu trebuie compilat):
build:
[TAB]$(CC) -o $(TARGET) zeus.o secondstep.o
[TAB]./$$(TARGET)

run:
[TAB]./$$(TARGET)

clean:
[TAB]rm -f $(TARGET) $(filter-out zeus.o,$(wildcard *.o))
# $(wildcard *.o) = toate .o-urile
# $(filter-out zeus.o,...) = exclude zeus.o din lista
# rm -f = force (fara eroare daca nu exista)

# Rulare:
make build
make run
make clean
```

■ **ATENȚIE:** *clean* șterge *.o* și executabile, dar **NU** *zeus.o*! Testeaza cu '*make clean*' și verifică '*ls*'.

4c. Regula pack — arhiva tar.gz

Makefile 4c

```
CC = gcc
TARGET = acadnet2025

.PHONY: build run clean pack # reguli care nu produc fisiere cu acel nume

pack:
[TAB]tar -czvf acadnet_rules.tar.gz Makefile $(wildcard *.c) $(wildcard *.h)
# $(wildcard *.c) = toate fisierele .c din directorul curent
# $(wildcard *.h) = toate fisierele .h
# tar -czvf = create + gzip + verbose + fisier

# Verificare:
make pack
tar -tf acadnet_rules.tar.gz # listeaza continutul

# IMPORTANT: cerinta spune sa te asiguri ca fisierul 'pack' nu e sters.
# .PHONY: pack rezolva asta -- fara .PHONY, daca exista un fisier 'pack',
# make considera ca regula e deja realizata si nu o ruleaza!
```

■ **TIP:** *.PHONY* declară reguli care nu corespund unor fișiere reale. Fără el, dacă există un fișier '*clean*' sau '*pack*', *make* va fi confuz.

Gestionarea Permisiiunilor

chmod, chown, groupadd, useradd, permisiuni octale

Sistemul de permisiuni Linux

Fiecare fisier/director are trei categorii de utilizatori: **owner** (proprietarul), **group** (grupul asociat) si **others** (toti ceilalti). Fiecare categorie poate avea dreptul de **read** (citire=4), **write** (scriere=2) si **execute** (executare=1). Suma acestora da numarul octal.

Logica permisiunilor

```
# rwx = 4+2+1 = 7
# rw- = 4+2+0 = 6
# r-x = 4+0+1 = 5
# r-- = 4+0+0 = 4
# --- = 0+0+0 = 0

# chmod 755 fisier = rwxr-xr-x
# 7 5 5
# owner group others

# Comanda: ls -la afiseaza permisiunile:
# drwxr-xr-x 2 student student 4096 Mai 3 10:00 AcadNet
# | | | |
# tip+perm link owner dimensiune
# d=dir -=fisier l=symlink
```

Octal	Simbolic	Semnificatie	Caz tipic
777	rwxrwxrwx	Toti au totul	public — acces complet
775	rwxrwxr-x	Owner+grup scriu, altii doar citesc+executa	scripturi de grup
770	rwxrwx---	Altii nu au deloc acces	director secret pentru grup
755	rwxr-xr-x	Owner scrie, altii doar citesc+executa	directoare publice standard
644	rw-r--r--	Owner scrie, altii doar citesc	fisiere text publice
600	rw-----	Doar owner-ul citeste/scrie	chei SSH, config privat
400	r-----	Doar owner-ul citeste (read-only)	fisiere de config critice
000	-----	Nimeni nu are acces	blocare completa

5c. Modificare permisiuni directoare si fisiere

chmod pentru database

```
# public – acces COMPLET pentru TOTI (777):
chmod 777 database/public
chmod 777 database/public/info
# sau recursiv pentru tot ce e inaintu:
chmod -R 777 database/public

# secret – acces doar owner + grup (770):
chmod 770 database/secret
chmod 770 database/secret/info

# config – DOAR owner poate citi/scrie (600):
touch database/secret/config
chmod 600 database/secret/config

# Verificare completa:
ls -la database/
ls -la database/secret/
ls -la database/public/
stat database/secret/config # detalii complete
```

5d. Gestionarea userilor si grupurilor

useradd, groupadd, chown

```
# ■■■ GRUPURI ■■■■  
sudo groupadd dbadmin # creeaza grupul  
getent group dbadmin # verifica grupul  
cat /etc/group | grep dbadmin # alternativa  
  
# ■■■ UTILIZATORI ■■■■  
sudo useradd -m -G dbadmin dbowner  
# -m = creeaza home directory (/home/dbowner)  
# -G dbadmin = adauga in grupul dbadmin (grupul secundar)  
  
sudo useradd -m dbuser # fara grup special  
  
# ■■■ ATRIBUIRE DIRECTOR CATRE GRUP ■■■■  
sudo chown -R dbowner:dbadmin database/secret # owner=dbowner, grup=dbadmin  
sudo chown dbowner:dbadmin database/secret/config  
  
# ■■■ VERIFICARE ■■■■  
id dbowner # uid, gid si grupuri  
id dbuser  
ls -la database/ # owner si grup pt fiecare  
groups dbowner # grupurile userului  
  
# Setare parola (daca e nevoie):  
sudo passwd dbowner  
sudo passwd dbuser  
  
# Adaugare user la grup (daca uiti -G la useradd):  
sudo usermod -aG dbadmin dbowner # -a = append, -G = grup
```

■ **ATENȚIE:** Cerinta spune 'dbadmi' (fara n la final) — citeste cerinta cu atentie! Poate fi intentionat sau typo.

systemd-analyze, blame, sort, awk

Ce este systemd?

systemd este sistemul de init modern folosit de majoritatea distribuțiilor Linux (Ubuntu, Debian, Fedora, etc.). El pornește toate serviciile la boot și le gestionează. **systemd-analyze** ne permite să vedem cât a durat fiecare etapă și fiecare serviciu.

6a. Afisare timpi de bootare

systemd-analyze de baza

```
# Timp total - rezumat:
systemd-analyze
# Output exemplu:
# Startup finished in 3.102s (kernel) + 12.543s (userspace) = 15.645s

# Detalii pe faze:
systemd-analyze time
# firmware + loader + kernel + initrd + userspace

# Grafic vizual in terminal (SVG):
systemd-analyze plot > boot.svg # export grafic
```

6b. Servicii ordonate crescator dupa timp

systemd-analyze blame listeaza serviciile ordonate **descrescator** dupa timpul de pornire (cel mai lent primul). Noi vrem ordinea **crescatoare** (cel mai rapid primul), deci trebuie sa inversam sau sa re-sortam.

blame ordonat crescator

```
# blame - listaaza descrescator (implicit):
systemd-analyze blame
# ex output:
# 12.543s apt-daily.service
# 3.201s NetworkManager.service
# 1.045s snapd.service

# ■■■■ ORDONARE CRESCATOARE ■■■■
systemd-analyze blame | sort -k1 -h
# -k1 = sorteaza dupa coloana 1 (timpul)
# -h = human-numeric (intelege 1ms, 1s, 1min)

# Alternativ - inversare cu tac:
systemd-analyze blame | tac # tac = cat inversat (ultima linie prima)
```

6c. Doar numele serviciilor, fara duplicate de timp

Daca mai multe servicii au acelasi timp, afisam doar primul. Folosim **awk** cu un array asociativ pentru a retine timpii vazuti.

Filtrare duplicate cu awk

```
# Comanda completa:
systemd-analyze blame | sort -k1 -h | awk '!seen[$1]++ {print $2}'

# Defalcăt:
# 1. systemd-analyze blame -> lista servicii (desc)
# 2. sort -k1 -h -> re-sorteaza crescator dupa timp
# 3. awk '!seen[$1]++ {print $2}'
# - $1 = prima coloana (timpul)
# - seen[$1] = retine daca am mai vazut acest timp
# - !seen[$1]++ = True doar la PRIMA aparitie a timpului
# - {print $2} = afiseaza a doua coloana (numele serviciului)

# Varianta extinsa cu explicatii inline:
systemd-analyze blame | tac | awk '{time=$1; name=$2} !seen[time]++ {print name}'
```

■ TIP: '!seen[\$1]++' este un pattern clasic awk pentru eliminarea duplicatelor. Memoreaza-l!

Folosirea Git

config, init, add, commit, branch, merge, amend

Ce este Git?

Git este un sistem de **versionare distribuita**. Permite salvarea istoricului modificarilor unui proiect, revenirea la versiuni anterioare si lucrul colaborativ. Fiecare 'snapshot' al proiectului se numeste **commit**. Ramurile de dezvoltare se numesc **branch-uri**.

Fluxul de lucru de baza in Git:

Fluxul Git

```
fisiere modificate --> git add --> Staging Area --> git commit --> Repository  
(Working Dir) (istoric)
```

7a. Configurare globala

Configuratia globala Git se salveaza in `~/.gitconfig`. Aceasta identificare apare in fiecare commit pe care il faci.

Configurare Git

```
# Configureaza identitatea:  
git config --global user.name 'student'  
git config --global user.email 'student@acadnet.ro'  
  
# Optional - editor implicit:  
git config --global core.editor nano  
  
# Verificare:  
git config --global --list  
cat ~/.gitconfig
```

7b. Initializare repo + copiere passwd

Init repo Git

```
# Creeaza directorul si initializeaza:
mkdir acadnet2025
cd acadnet2025
git init
# Output: Initialized empty Git repository in ../acadnet2025/.git/

# Gaseste fisierul passwd:
find / -name 'passwd' -not -path '*/proc/*' 2>/dev/null | head -5
# Cel mai probabil e /etc/passwd

# Copiaza-l in repo:
cp /etc/passwd .

# Adauga si fa primul commit:
git add passwd
git commit -m "Add passwd file"

# Verificare:
git log --oneline
git status
```

7c. Branch networking + ifconfig + commit

Branch + commit

```
# Creeaza branch-ul si fa switch pe el:
git checkout -b networking
# -b = creeaza branch nou (-b = new branch)
# Dupa aceasta comanda, esti pe branch-ul 'networking'

# Salveaza output-ul ifconfig in fisierul 'ports':
ifconfig > ports
# Daca ifconfig nu e disponibil (Ubuntu modern):
ip addr > ports
ip addr show > ports

# Adauga si comiteaza:
git add ports
git commit -m "Suspicious ports"

# Verificare:
git log --oneline
git branch # arata branch-ul curent cu *
git diff HEAD~1 # ce s-a schimbat fata de commit-ul anterior
```

7d. Modificarea mesajului ultimului commit

git commit --amend modifica ultimul commit. Poate schimba mesajul si/sau continutul. **Atentie:** nu folosi --amend pe commit-uri deja impinse pe un server remote (rescrie istoria).

git commit --amend

Modifica mesajul ultimului commit:

git commit --amend -m "Not suspicious ports"

-m = noul mesaj direct in comanda

Alternativ – deschide editorul (nano/vim) pentru modificare:

git commit --amend

In editorul care se deschide: modifica mesajul, salveaza si iese

Verificare:

git log --oneline

git log -1 --pretty=%B # afiseaza doar mesajul ultimului commit

Alte comenzi Git utile:

git log --oneline --graph --all # grafic al tuturor branch-urilor

git diff # modificari nestagiate

git diff --staged # modificari stagiate (dupa add)

git reset --hard HEAD~1 # anuleaza ultimul commit (PERICULOS)

git stash # salveaza temporar modificarile

Provocarea Batman vs. Joker

PATH hijacking, LD_PRELOAD, /etc/profile.d/

Intelegerea problemei

Cerinta: comanda **id** sa afiseze informatiile root pentru orice user, indiferent cine o ruleaza. Restrictiile: fara alias, fara modificare /usr/bin/id, fara scripturi evidente in /bin/, trebuie sa fie general (toti userii) si reversibil.

Cum functioneaza cautarea comenzilor in Linux?

Cum functioneaza PATH

```
# Cand scrii 'id' in terminal, bash cauta in PATH:
echo $PATH
# ex: /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Bash cauta stanga la dreapta si ruleaza PRIMUL 'id' gasit.
# Daca punem un 'id' fals INAINTE de /usr/bin in PATH, il ruleaza pe acela!

# Unde e comanda 'id' originala:
which id
# Output: /usr/bin/id
```

Solutia — PATH Hijacking cu /etc/profile.d/

Fisierele din **/etc/profile.d/** sunt surse automat de bash la login pentru **toti utilizatorii**. Deci daca acolo adaugam un PATH cu un director custom primul, toti userii vor rula 'id'-ul nostru.

1 Creeaza un director pentru comanda falsa:

Alegem un loc mai putin evident:

```
sudo mkdir -p /usr/local/bin/custom
```

2 Scrie scriptul 'id' fals:

Scriptul va apela /usr/bin/id cu argumentul 'root':

```
sudo nano /usr/local/bin/custom/id

# Continut fisier (scrie in nano):
#!/bin/bash
/usr/bin/id root

# Salveaza: Ctrl+O → Enter → Ctrl+X
```

3 Fa scriptul executabil:

```
sudo chmod +x /usr/local/bin/custom/id
```

4 Adauga PATH global in /etc/profile.d/:

Creeaza un fisier nou in profile.d — se aplica la toti userii:

```
sudo nano /etc/profile.d/joker.sh

# Continut (scrie in nano):
export PATH=/usr/local/bin/custom:$PATH

# Salveaza: Ctrl+O → Enter → Ctrl+X

# Fa-l executabil:
sudo chmod +x /etc/profile.d/joker.sh
```

5 Aplica si testeaza:

```
source /etc/profile.d/joker.sh
id
# Ar trebui sa apara: uid=0(root) gid=0(root) groups=0(root)

# Testeaza si cu alt user (daca ai):
su - dbuser -c 'id'
```

6 REVERSIBIL — cum anulezi:

Stergerea fisierelor si re-logarea anuleaza totul:

```
sudo rm /etc/profile.d/joker.sh
sudo rm /usr/local/bin/custom/id
# Logout + login (sau deschide terminal nou)
```

Solutia alternativa — LD_PRELOAD (nivel avansat)

LD_PRELOAD permite incarcarea unei biblioteci dinamice (.so) inainte de orice alta biblioteca. Putem suprascrie functiile **getuid()** si **getgid()** sa returneze 0 (root) mereu. Dezavantaj: nu functioneaza pe binare SUID.

LD_PRELOAD (avansat)

```
# Creeaza fisierul C:
nano /tmp/fakeid.c

# Continut fakeid.c:
#include <sys/types.h>
uid_t getuid(void) { return 0; }
uid_t geteuid(void) { return 0; }
gid_t getgid(void) { return 0; }
gid_t getegid(void) { return 0; }

# Compileaza ca shared library:
gcc -shared -fPIC -o /usr/local/lib/fakeid.so /tmp/fakeid.c

# Seteaza global in /etc/environment:
sudo nano /etc/environment
# Adauga linia:
LD_PRELOAD=/usr/local/lib/fakeid.so

# Reversibil: sterge linia din /etc/environment
```

■ **DE RETINUT:** La concurs, solutia **PATH Hijacking** e mai simpla si mai sigura. **LD_PRELOAD** poate fi blocat pe unele sisteme.